

# DATA 260 - Homework 3

**Student:** Sanjana Thummalapalli

**SID4:** 7801

**Domain:** Clinical Trial Listings

**GitHub Repository:**

<https://github.com/sanjana-glitch-art/data260-7801>

## 1. Personal Configuration

|                 |                         |
|-----------------|-------------------------|
| Student Name    | Sanjana Thummalapalli   |
| SID4            | 7801                    |
| PORT_BASE       | 8601                    |
| PREFIX          | s7801                   |
| SEED            | 7801                    |
| VERIFY_SEED     | 267801                  |
| DOMAIN_ID       | 1                       |
| Assigned Domain | Clinical Trial Listings |

## 2. Homework Overview

This homework extended the clinical-trial listing application developed in the previous assignments.

Part 1 added a FastAPI authentication system with login, logout, session management, an idle timeout, protected routes, secure cookie settings, Jinja templates, and Bootstrap styling.

Part 2 implemented a retrieval-only RAG experiment using LlamaIndex. Three chunking techniques were compared:

1. Token-based chunking
2. Semantic chunking
3. Sentence-window chunking

The comparison used a local clinical-trial corpus built from public FDA documents. The experiment measured chunk counts, chunk lengths, cosine similarity, source recall, and retrieval latency.

## Part 1 - FastAPI Authentication

### 3. Authentication Application Structure

The authentication implementation uses a separate FastAPI APIRouter in code/auth.py. The router is connected to the existing application in code/main.py.

The application contains these routes:

| Route      | Method | Purpose                                   |
|------------|--------|-------------------------------------------|
| /          | GET    | Public home page                          |
| /login     | GET    | Display the login form                    |
| /login     | POST   | Validate credentials and create a session |
| /dashboard | GET    | Protected authenticated dashboard         |
| /logout    | GET    | Clear the session and return home         |
| /trials    | GET    | Existing clinical-trial application       |
| /docs      | GET    | FastAPI documentation                     |

## Router Registration Code

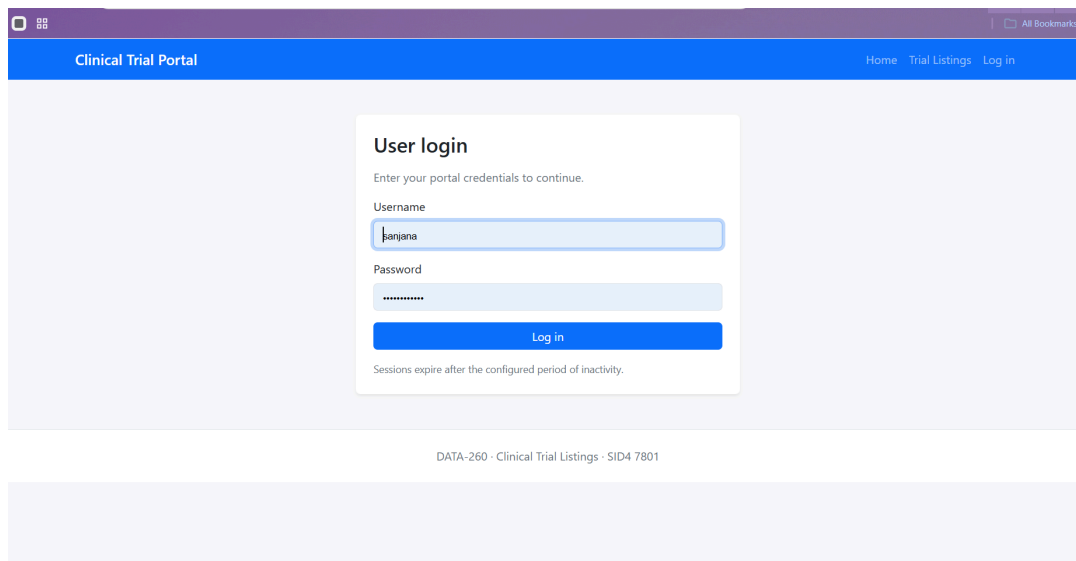
```
run_hw3_retrieval.py X main.py X
code > main.py > ...
51 class TrialCreate(BaseModel):
52     """Values accepted when creating a trial."""
53
54     trial_title: str = Field(min_length=1)
55     sponsor_name: str = Field(min_length=1)
56     submitter_email: str = ""
57     trial_description: str = ""
58     trial_phase: str = "Phase I"
59
60
61 class TrialUpdate(BaseModel):
62     """Values accepted when updating a trial."""
63
64     trial_title: str = Field(min_length=1)
65     sponsor_name: str = Field(min_length=1)
66
67
68 app = FastAPI(
69     title="Clinical Trial Listing API",
70     version="3.0.0"
71 )
72
73 app.add_middleware(
74     SessionMiddleware,
75     secret_key=SESSION_SECRET,
76     session_cookie=f"{PREFIX}_session",
77     max_age=SESSION_IDLE_SECONDS,
78     same_site="lax",
79     https_only=True,
80     path="/"
81 )
```

## 4. Public Home Page

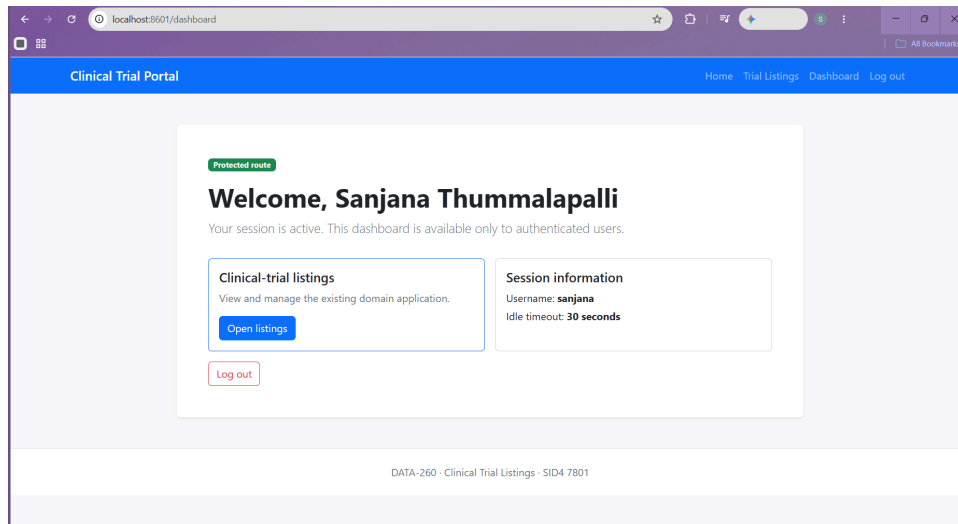
The home page displays a welcome message for the Clinical Trial Listing Portal. When the user is logged out, it displays a link to the login page. When the user is authenticated, it displays links to the dashboard, trial listings, and logout.

## Home Route Code

```
run_hw3_retrieval.py X auth.py X
code > auth.py > home_page
100 def credentials_are_valid(
116     return (
117         username_matches
118         and password_matches
119     )
120
121
122 @router.get(
123     "/",
124     response_class=HTMLResponse
125 )
126 async def home_page(
127     request: Request
128 ) -> HTMLResponse:
129     """Display the application home page."""
130
131     user = session_user(request)
132
133     return templates.TemplateResponse(
134         request=request,
135         name="home.html",
136         context={
137             "user": user,
138             "idle_timeout": SESSION_IDLE_SECONDS
139         }
140     )
```



**Figure 1.** Public Clinical Trial Listing Portal home page before login.



*Figure 2. Home page displaying authenticated navigation options.*

## 5. Login Page and Credential Validation

The login page accepts a username and password. The local demonstration credentials are:

Username: sanjana

Password: Data260!7801

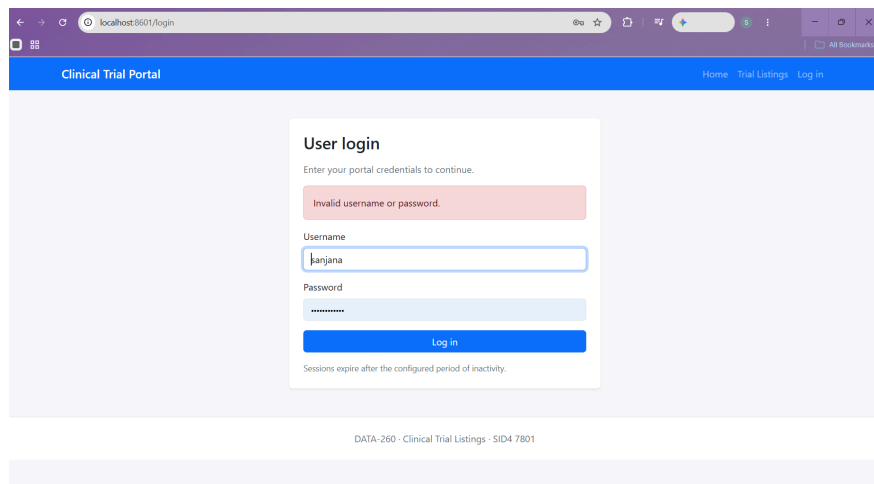
Credentials are compared using `secrets.compare_digest`. This is preferable to ordinary string equality for credential comparisons.

### Credential Validation Code

```
run_hw3_retrieval.py x auth.py x
code > auth.py > credentials_are_valid
65 def session_user(
96
97     return user
98
99
100 def credentials_are_valid(
101     username: str,
102     password: str
103 ) -> bool:
104     """Safely compare the submitted credentials."""
105
106     username_matches = secrets.compare_digest(
107         username,
108         DEMO_USERNAME
109     )
110
111     password_matches = secrets.compare_digest(
112         password,
113         DEMO_PASSWORD
114     )
115
116     return (
117         username_matches
118         and password_matches
119     )
120
```

### Login Processing Code

```
run_hw3_retrial.py X auth.py X
ode > auth.py | process_login
147 async def login_page(
178 )
179
180
181 @router.post(
182     "/login",
183     response_class=HTMLResponse
184 )
185 async def process_login(
186     request: Request,
187     username: str = Form(...),
188     password: str = Form(...)
189 ) -> Response:
190     """Validate credentials and create a session."""
191
192     cleaned_username = username.strip()
193
194     if not credentials_are_valid(
195         cleaned_username,
196         password
197     ):
198         request.session.clear()
199
200     return templates.TemplateResponse(
201         request=request,
202         name="login.html",
203         context={
204             "user": None,
205             "error": (
206                 "Invalid username or password."
207             ),
208             "message": None,
209             "username": cleaned_username
210         },
211         status_code=(
212             status.HTTP_401_UNAUTHORIZED
213         )
214     )
215
216     request.session.clear()
217
218     request.session["user"] = {
219         "email": email,
220         "password": password,
221         "username": username,
222         "is_authenticated": True
223     }
224     return Response(status.HTTP_200_OK)
```



*Figure 3. Bootstrap alert displayed after an invalid login attempt.*

## 6. Protected Dashboard

The dashboard is protected by checking the session before rendering the template. A user who has not logged in is redirected to /login. A user whose session expired is redirected to /login?expired=1.

## Protected Dashboard Code

```

run_hw3_retrieval.py X auth.py X
code > auth.py > dashboard_page
233 @router.get(
234     "/dashboard",
235     response_class=HTMLResponse
236 )
237 async def dashboard_page(
238     request: Request
239 ) -> Response:
240     """Display the protected dashboard."""
241
242     # Check whether a session existed before session_user()
243     # potentially clears it because it expired.
244     had_session = bool(
245         request.session.get("user")
246     )
247
248     user = session_user(request)
249
250     if user is None:
251         request.session.clear()
252
253         redirect_url = {
254             "/login?expired=1"
255             if had_session
256             else "/login"
257         }
258
259         return RedirectResponse(
260             url=redirect_url,
261             status_code=(
262                 status.HTTP_303_SEE_OTHER
263             )
264         )
265
266     return templates.TemplateResponse(
267         request=request,
268         name="dashboard.html",
269         context={
270             "user": user,
271             "idle_timeout": SESSION_IDLE_SECONDS
272         }
273     )
274

```

## 7. Logout Protection

The logout route clears all session information and redirects the user to the public home page.

### Logout Code

```

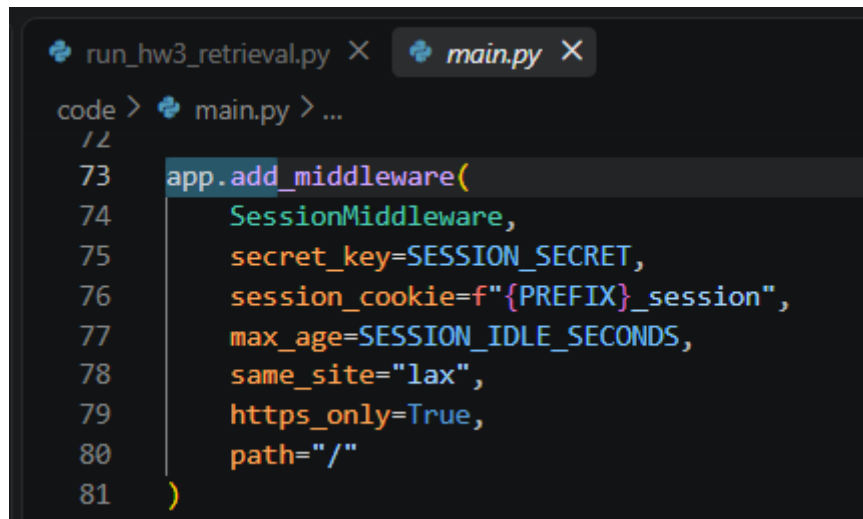
run_hw3_retrieval.py X auth.py X
code > auth.py > logout
237 async def dashboard_page(
267     request=request,
268     name="dashboard.html",
269     context={
270         "user": user,
271         "idle_timeout": SESSION_IDLE_SECONDS
272     }
273 )
274
275
276 @router.get("/logout")
277 async def logout(
278     request: Request
279 ) -> RedirectResponse:
280     """Destroy the session and redirect home."""
281
282     request.session.clear()
283
284     return RedirectResponse(
285         url="/",
286         status_code=status.HTTP_303_SEE_OTHER
287     )

```

## 8. Session Management and Secure Cookie

Starlette's `SessionMiddleware` manages the authenticated session. The cookie uses a student-specific name based on the assigned prefix.

### Session Middleware Code



```
run_hw3_retrieval.py × main.py ×
code > main.py > ...
12
73 app.add_middleware(
74     SessionMiddleware,
75     secret_key=SESSION_SECRET,
76     session_cookie=f"{PREFIX}_session",
77     max_age=SESSION_IDLE_SECONDS,
78     same_site="lax",
79     https_only=True,
80     path="/"
81 )
```

The resulting cookie name is:

s7801\_session

The cookie has these security attributes:

| Attribute    | Purpose                                                |
|--------------|--------------------------------------------------------|
| HttpOnly     | Prevents JavaScript from reading the cookie            |
| Secure       | Sends the cookie only through a secure browser context |
| SameSite=Lax | Reduces cross-site request risks                       |
| Path=/       | Makes the session available across application routes  |
| Max-Age      | Limits the cookie lifetime                             |

## 9. Idle Session Timeout

The application records the time of the user's last valid activity. Every protected request compares the current time with the saved activity time.

### Idle Timeout Code

```
def session_user(
    request: Request
) -> dict[str, Any] | None:
    user = request.session.get("user")

    last_activity = request.session.get(
```

```

        "last_activity"
    )

    if not user or not last_activity:
        return None

    idle_seconds = (
        current_timestamp()
        - int(last_activity)
    )

    if idle_seconds > SESSION_IDLE_SECONDS:
        request.session.clear()
        return None

    request.session["last_activity"] = (
        current_timestamp()
    )

    return user

```

For testing, the idle timeout was temporarily set to 30 seconds. After remaining inactive for more than 30 seconds, refreshing the dashboard redirected the browser to:

```
/login?expired=1
```

The expired session could not be reused.

## 10. Bootstrap Styling and Templates

Bootstrap was used to create a responsive interface. The application uses:

- Navbar navigation
- Cards for page content and forms
- Buttons for actions
- Alerts for errors and messages
- Responsive grid columns
- Responsive navigation

The template directory contains:

```

code/templates/
├── base.html
├── dashboard.html
├── home.html
└── login.html

```

## Part 2 - LlamaIndex Retrieval-Only RAG

### 11. Clinical-Trial Corpus

The graded corpus contains local text snapshots of three public FDA clinical-trial guidance documents.

| Corpus file                      | Size                 |
|----------------------------------|----------------------|
| fda_informed_consent.txt         | 194,915 bytes        |
| fda_risk_based_monitoring.txt    | 67,922 bytes         |
| fda_clinical_trial_endpoints.txt | 87,899 bytes         |
| <b>Total</b>                     | <b>350,736 bytes</b> |

The total corpus size is 350,736 bytes, which exceeds the required minimum of 200,000 bytes.

The source URLs and access information are documented in:

reports/hw03/SOURCES.md

The local filenames, sizes, and SHA-256 hashes are stored in:

reports/hw03/CORPUS\_MANIFEST.json

#### Corpus Hashes

| File                             | SHA-256                                                          |
|----------------------------------|------------------------------------------------------------------|
| fda_informed_consent.txt         | 587220824b10fea4fdf29b34a8fb23f5cba294de12a199a7010ef33211f7ec7f |
| fda_risk_based_monitoring.txt    | 2935b7f1590c7a32f0475088c7a14c35b7ba2e3ebbd3fbad78ceeb39e28b7732 |
| fda_clinical_trial_endpoints.txt | d8d26a1eaa1b8697f9203cf24a7e95be7361425b3e81f83373439a3d892f8bc0 |

### 12. Question Preregistration

Five graded questions were written with expected answers and expected source files before retrieval results were generated.

The original questions were committed in Git commit: fb6516b

This prevents expected answers from being rewritten after observing the results.

An additional sixth question was added and committed before rerunning the experiment because the first five questions did not produce a false-positive source mismatch.

The questions are stored in:

```
! questions.yaml X
reports > hw03 > ! questions.yaml

1  domain_id: 1
2  domain: "Clinical Trial Listings"
3  student: "Sanjana Thummalapalli"
4  sid4: 7801
5
6  questions:
7    - id: q1
8      question: >
9        Why is informed consent considered an ongoing process
10       rather than only obtaining a participant's signature?
11     expected_answer: >
12       Informed consent requires giving prospective participants
13       adequate information, helping them understand it, allowing
14       questions and time for consideration, obtaining voluntary
15       agreement, and continuing to provide relevant information
16       throughout the clinical investigation.
17     expected_source_file: "fda_informed_consent.txt"
18     source_specific: true
19
20   - id: q2
21     question: >
22       What protections must be present when an investigator
23       obtains informed consent from a clinical-trial participant?
24     expected_answer: >
25       Consent must be obtained under circumstances that provide
26       sufficient opportunity to decide, minimize coercion or undue
27       influence, use understandable language, and avoid language
28       that waives the participant's legal rights or releases others
29       from liability for negligence.
30     expected_source_file: "fda_informed_consent.txt"
31     source_specific: true
```

### 13. Embedding Model and Vector Index

The experiment used:

sentence-transformers/all-MiniLM-L6-v2

The embedding dimension was 384. Each technique used an in-memory LlamaIndex VectorStoreIndex backed by SimpleVectorStore.

#### In-Memory Index Code

```
def build_index(
    nodes: list[BaseNode],
    embed_model: HuggingFaceEmbedding
) -> VectorStoreIndex:
    vector_store = SimpleVectorStore()

    storage_context = (
        StorageContext.from_defaults(
            vector_store=vector_store
        )
    )

    return VectorStoreIndex(
        nodes,
```

```

        storage_context=storage_context,
        embed_model=embed_model,
        show_progress=True
    )

```

## 14. Token-Based Chunking

Token chunking used:

- Chunk size: 512 tokens
- Chunk overlap: 64 tokens

### Token Chunking Code

```

token_splitter = TokenTextSplitter(
    chunk_size=512,
    chunk_overlap=64
)

```

```

token_nodes = (
    token_splitter
    .get_nodes_from_documents(
        documents,
        show_progress=True
    )
)

```

Token splitting produced 162 chunks with an average returned length of 2,432.29 characters.

## 15. Semantic Chunking

Semantic chunking used:

- Buffer size: 1
- Breakpoint percentile threshold: 95
- The shared MiniLM embedding model

### Semantic Chunking Code

```

semantic_splitter = (
    SemanticSplitterNodeParser(
        buffer_size=1,
        breakpoint_percentile_threshold=95,
        embed_model=embed_model
    )
)

```

```

semantic_nodes = (
    semantic_splitter
    .get_nodes_from_documents(

```

```

        documents,
        show_progress=True
    )
)

```

Semantic splitting produced 90 chunks with an average returned length of 3,824.51 characters.

## 16. Sentence-Window Chunking

Sentence-window chunking indexed individual sentences and attached three neighboring sentences as metadata on each side.

### Sentence-Window Code

```

sentence_window_splitter = (
    SentenceWindowNodeParser.from_defaults(
        window_size=3,
        window_metadata_key="window",
        original_text_metadata_key=(
            "original_sentence"
        )
    )
)

```

```

sentence_window_nodes = (
    sentence_window_splitter
    .get_nodes_from_documents(
        documents,
        show_progress=True
    )
)

```

Sentence-window splitting produced 1,705 nodes with an average returned context length of 1,412.98 characters.

## 17. Retrieval-Only Measurement Function

For each question, the experiment:

1. Computed the query embedding.
2. Printed its dimension.
3. Printed its first eight values.
4. Retrieved the top five nodes.
5. Recorded the vector-store score.
6. Explicitly embedded each returned chunk.
7. Calculated cosine similarity.
8. Recorded chunk length.
9. Recorded the source file.
10. Printed a 160-character preview.
11. Timed the similarity search.

## 12. Calculated source-based Recall@5.

### Cosine Similarity Code

```
def cosine_similarity(  
    first: list[float] | np.ndarray,  
    second: list[float] | np.ndarray  
) -> float:  
    first_array = np.asarray(  
        first,  
        dtype=np.float64  
    )  
  
    second_array = np.asarray(  
        second,  
        dtype=np.float64  
    )  
  
    denominator = (  
        np.linalg.norm(first_array)  
        * np.linalg.norm(second_array)  
    )  
  
    if denominator == 0:  
        return 0.0  
  
    return float(  
        np.dot(  
            first_array,  
            second_array  
        )  
        / denominator  
    )
```

The complete machine-readable results are stored in:

```
reports/hw03/raw/retrieval_results.jsonl  
reports/hw03/raw/retrieval_results.csv  
reports/hw03/raw/retrieval_metrics.json
```

## 18. Retrieval Results

The metrics below are averages across all six evaluated questions.

| Technique | Chunks | Average chunk length | Mean top-1 cosine | Mean@5 cosine | Recall@5 | Mean latency |
|-----------|--------|----------------------|-------------------|---------------|----------|--------------|
| Token     | 162    | 2,432.29             | 0.6765            | 0.6271        | 100%     | 27.67 ms     |

|                     |       |          |        |        |      |          |
|---------------------|-------|----------|--------|--------|------|----------|
| Semantic            | 90    | 3,824.51 | 0.6155 | 0.5833 | 100% | 22.00 ms |
| Sentence-win<br>dow | 1,705 | 1,412.98 | 0.6842 | 0.6084 | 100% | 86.13 ms |

## 19. False-Positive Retrieval

The first five questions did not produce a source mismatch. Following the assignment instructions, I added a sixth question:

What should a clinical-trial sponsor do to protect participants and ensure that study results are reliable?

The expected answer concerned risk-based monitoring and the expected source was:

fda\_risk\_based\_monitoring.txt

Token retrieval returned a rank-five result from:

fda\_informed\_consent.txt

The result had a cosine similarity of 0.5353. Its preview discussed clinical investigations and standards of care, but it did not explain the required risk-based monitoring approach.

| Field             | Result                        |
|-------------------|-------------------------------|
| Technique         | Token                         |
| Question          | q6                            |
| Rank              | 5                             |
| Cosine similarity | 0.5353                        |
| Expected source   | fda_risk_based_monitoring.txt |
| Retrieved source  | fda_informed_consent.txt      |
| Source match      | False                         |

The embedding model probably considered this passage relevant because both the question and passage used related concepts such as clinical trials, sponsors, participants, investigations, safety, and standards of care. Embedding similarity measures general semantic relatedness. It does not guarantee that the passage contains the specific answer.

This result demonstrates that a relatively confident similarity score should not automatically be treated as evidence that a retrieved passage answers the question.

## 20. Observations

Sentence-window chunking achieved the highest mean top-1 cosine similarity, at 0.6842. Indexing individual sentences made it easier to locate one highly focused passage. The attached neighboring sentences provided additional context. However, this method created 1,705 nodes and had the highest mean retrieval latency at 86.13 milliseconds.

Token chunking achieved the highest Mean@5 cosine similarity, at 0.6271. It created only 162 chunks and had a mean latency of 27.67 milliseconds. The fixed-size chunks provided enough surrounding context without creating an excessively large index.

Semantic chunking created only 90 chunks and had the lowest mean latency at 22 milliseconds. However, it also produced the lowest top-1 and Mean@5 similarities. Its larger chunks often combined several related ideas, making the retrieved passages less focused on a specific question.

All three techniques achieved 100% source-based Recall@5. Therefore, the major differences were similarity, chunk granularity, index size, and latency.

## 21. Verification

The self-check script is located at:

code/verify\_hw3.py

It checks:

- Required files
- Python version
- Dependencies
- Authentication implementation
- FastAPI routes
- Credential validation
- Bootstrap templates
- Corpus sizes and SHA-256 hashes
- Questions
- Git preregistration order
- Retrieval run counts
- Required metrics
- False-positive evidence
- Run-log evidence

The final result was:

Overall passed: True

The machine-readable verification output is stored in:

reports/hw03/verification.json